Augmented Reality Module

1. Overview

The AR module allows users to spectate ongoing duels of the Arcane Gambit game in real-time using Augmented Reality. It connects to a MongoDB real-time database to retrieve the latest game state and visualizes characters, their health bars, and attack animations in the physical world through Unity's AR Foundation framework.

The goal is to create a smooth and immersive spectator experience on mobile devices that blends live gameplay with AR visuals, synced with backend updates.

2. Technologies Used

- **Unity 2022.3.50f1 -** Game engine used to build the AR experience
- **AR Foundation + ARCore -** For device tracking and AR rendering
- **WebSocket++ -** Used for real-time game state communication

3. Features

- **Live Duello Spectating**
    i. Retrieves real-time duello data (player usernames, model names, players' health values, attack information) from server.
    ii. Renders this data as 3D elements in AR space.
- **Player UI (Username + Bars)**
    i. Displays each player's username and health bar on top of their character model.
- **Attack Visualizations**
    i. Shows temporary visual effects (e.g., fireball, lightning) in AR when an attack is triggered.
- **Prefab-Based Character Spawning**
    i. Uses Unity prefabs to spawn characters in specific AR positions
    ii. Can vary based on the player's class (warrior, mage, etc.)

4. Code Structure & Classes

i. WebSocketPlugin.cpp (C++ side – Unity Plugin Wrapper)

Handles backend communication through WebSocket++. This wrapper fetches the latest game state and provides native API bindings for Unity to consume via a C++ SO (shared objects) file.

- Core Components

```
struct PlayerState {
    std::string id;
    std::string username;
    std::string modelName;
    int health;
    std::string status;
    std::string attackName = "";
    std::string attackDamage = 0;
};

struct GameState {
    std::vector<PlayerData> players;
    std::string gameStatus; //started", "ongoing", "finished"
    std::string turnPlayerId;
};

extern "C" {
            void InitializeWebSocket(const char *);
            void SendMsg(const char *);
            void CloseWebSocket();
            bool IsConnected();
}
```

- Used for:
    - Opening/closing WebSocket connection from Unity.
    - Receiving and parsing real-time game data.
    - Feeding Unity with structured player and attack information.
    - Managing WebSocket state and processing incoming messages.
    - Sending JSON payloads to backend (if needed).

ii. WebSocketBridge.cs

TO BE WRITTEN

iii. MainSceneManager.cs

Oversees the core AR gameplay setup and interaction, such as placing the game arena and managing multiplayer sessions.

- Key Features:
    - Uses LeanTouch to allow users to easily manipulate game arena by scaling and moving it in the environment.
    - Handles button clicks for actions like joining a room.
    - Coordinates session settings with the *WebSocketBridge*.

- Core Components:

```
void PlaceArena(Vector2 screenPos);
```

iv. **GameController.cs**

Manages the spawning of players and the game logic, including player states, actions, and turn-based gameplay.

- Key Features:
  - Calculates positions within the arena floor for spawning players.
  - Spawns players using appropriate player prefabs and applies specific positions and rotations.
  - Dynamically finds the arena's floor object to ensure accurate player placement.
  - Handles player actions like attacks, damages, and animations.
  - Tracks and updates the current turn for the game.

- Core Components:

```
void UpdateGame();
void SpawnPlayers();
void SpawnPlayer(PlayerState player, Vector3 position, Quaternion
rotation);
void HandlePlayerStateChanged(PlayerState player, PlayerController
playerController);
```

v. **PlayerController.cs**

Manages each player's visual representation, health system, animations, and attack effects in the AR scene. It is directly controlled by the *GameController* and reacts to updates from the live game state.

- Key Features:
  - Scales and recolors a health bar based on current HP.
  - Plays animations and visual feedback when taking damage.
  - Instantiates appropriate attack VFX in front of the player.
  - Implements visual effects for player attacks using particle systems and smooth animations.

- Core components:

```
void SetHealth(int newHealth);
void TakeDamage(int damage);
void Attack(string attackName);
```

5. Detailed Implementation Description

- `WebSocketPlugin.cpp`

  These functions are exposed from the C++ side (SO) and consumed by Unity through native plugin bindings (e.g., `SOImport`).

  - `struct PlayerState`

`id`	Unique player identifier (matched `turnPlayerId`)
`username`	Display name of the player
`modelName`	The 3D model prefab name to load in Unity
`health`	Current health value
`status`	Current status ("idle", "attacking", "dead", etc.)
`attackName`	Name of the attack being performed (optional)
`attackDamage`	Damage value dealt by the current attack (optional)

  - `struct GameState`

`players`	List of all `PlayerState` objects in the match
`gameStatus`	Current game status
`turnPlayerId`	ID of the player whose turn is currently active

  - `InitializeConnection`
    - `Inputs: const char* serverUrl, int roomCode`
    - `Output: bool`
    - Purpose: Establishes a WebSocket++ connection to the game server using the specified URL and room code.
    - Usage Notes:
      - Should be called once when the AR scene initializes.
      - Room code should match a valid ongoing game session.

  - `ShutdownConnection`
    - `Inputs: void`
    - `Output: void`
    - Purpose: Gracefully closes the WebSocket connection and cleans up associated resources.
    - Usage Notes:
      - Call this when exiting the AR scene or shutting down the application.
      - Prevents memory leaks or dangling threads.

  - `ProcessEvents`

- Inputs: void
- Output: bool
- Purpose: Processes incoming WebSocket messages, handles internal message parsing, and updates the internal game state cache.
- Usage Notes:
  - Should be called once per frame (e.g., inside `Update()` in Unity) to ensure real-time updates.

- GetGameStateJson
  - Inputs: void
  - Output: const char* gameState
  - Purpose: Retrieves the latest game state as a JSON-formatted string.
  - Usage Notes:
    - Should be called after `ProcessEvents()` to get the latest data.
    - Must be parsed in Unity using `JsonUtility` or `Newtonsoft.Json`.

  - Format Example:

```
{
    "gameStatus": "ongoing",
    "currentTurnPlayerId": "p1",
    "players": [
        {
            "id": "p1",
            "username": "WarriorQueen",
            "health": 85,
            "modelName": "maria",
            "state": "attacking",
            "attackType": "fireball",
            "attackDamage": 15
        },
        {
            "id": "p2",
            "username": "Voldemort",
            "health": 102,
            "modelName": "pumpkinhulk",
            "state": "idle",
            "attackType": "",
            "attackDamage": 0
        }
```

```
    ]
}
```

- IsConnected
  - Inputs: void
  - Output: bool
  - Purpose: Checks the status of the WebSocket connection.
  - Usage Notes:
    - Useful for error handling and reconnect logic in Unity.

- WebSocketPlugin.h

Each function is exposed using Unity's native plugin interface macros:

```
UNITY_INTERFACE_EXPORT void UNITY_INTERFACE_API FunctionName(...);
```

This ensures:

- Compatibility with Unity's C# DllImport
- Cross-platform usage (Windows, Android, iOS with proper builds).

- SetMsgCallback
  - Inputs: const char* msg
  - Output: void
  - Purpose: Registers a C-style callback function that Unity will use to receive messages asynchronously from the WebSocket server.
  - Usage Notes:
    - This must be set **before** initializing the WebSocket connection.
    - Unity should define a C# delegate and pass it via Marshal.GetFunctionPointerForDelegate.
  - Format Example:

```
[UnmanagedFunctionPointer(CallingConvention.Cdecl)]
public delegate void
MessageCallback([MarshalAs(UnmanagedType.LPStr)] string msg);

[DllImport("WebSocketPlugin")]
private static extern void SetMsgCallback(IntPtr callback);
```

- InitializeWebSocket
  - Inputs: const char* url
  - Output: void

        - Purpose: Opens a WebSocket connection to the specified server URL.
        - Usage Notes:
            - Should be called after setting the message callback.
            - Connection success can be checked using `IsConnected()`.

    - `SendMsg`
        - `Inputs: const char* msg`
        - `Output: void`
        - Purpose: Sends a raw string message to the server over the active WebSocket connection.
        - Usage Notes:
            - Use to send actions or game-related commands if needed (e.g., ping, ready-check).
            - Can also be used for debugging with custom test payloads.

    - `CloseWebSocket`
        - `Inputs: void`
        - `Output: void`
        - Purpose: Closes the WebSocket connection and releases any allocated resources.
        - Usage Notes:
            - Must be called when leaving the AR scene or quitting the game to prevent socket leaks or dangling threads.

    - `IsConnected`
        - `Inputs: void`
        - `Output: bool`
        - Purpose: Returns whether the WebSocket connection is currently active and open.
        - Usage Notes:
            - Useful for retry/reconnect logic or to display connection status in the UI.

`NetworkManager.cs`

A Unity-side singleton or service class responsible for handling all networking operations in the AR spectator module. It abstracts the WebSocket plugin layer and provides:

- Reliable message queuing and sequential processing

- Latency tracking
- Connection health monitoring
- Network configuration adjustments (e.g., retries, timeouts)

- `QueueMsg`
  - `Inputs: Action msgAction`
  - `Output: void`
  - Purpose: Enqueues a message-related action for sequential processing on Unity's main thread.
  - Usage Notes:
    - Useful for thread safety: WebSocket callbacks occur on non-main threads, so UI and game logic updates must be enqueued and processed on Unity's main thread.
    - This function should be called from the native plugin's message callback.

- `ProcessMessageQueue`
  - `Inputs: void`
  - `Output: IEnumerator`
  - Purpose: Coroutine that processes all queued message actions one by one, executing them on the main thread.
  - Usage Notes:
    - Should be started once on game launch (e.g., `StartCoroutine(ProcessMessageQueue())`).
    - Prevents frame stutter by spacing out expensive operations.
    - Can use `yield return null` to wait one frame between executions if needed.

- `RecordMsgLatency`
  - `Inputs: float latency`
  - `Output: void`
  - Purpose: Records the latency of the last received message for monitoring performance and adapting the UI or retry policies.
  - Usage Notes:
    - Can be used to dynamically show "Connection unstable" messages.
    - Could be visualized in debug UI for dev builds.

- `AdjustNetworkSettings`
  - `Inputs: void`
  - `Output: void`

- Purpose: Adjusts connection parameters based on current network conditions or performance metrics.
- Usage Notes:
  - Can tune message frequency, retry intervals, ping timeouts, etc.
  - May use `RecordMsgLatency()` or failed reconnect attempts as input.
  - Should be invoked periodically or on significant status change.

- `MonitorConnection`
  - `Inputs: void`
  - `Output: IEnumerator`
  - Purpose: Coroutine that continuously checks the WebSocket connection status and triggers reconnection attempts if needed.
  - Usage Notes:
    - Should be run in parallel with message processing (`StartCoroutine(MonitorConnection())`).
    - If `WebSocketPlugin.IsConnected()` returns false for a threshold period:
    - Attempt to reconnect or show a warning.
    - Can call `InitializeWebSocket()` and re-register the callback when reconnecting.

`WebSocketManager.cs`

Handles low-level WebSocket communication inside Unity by interfacing with your native plugin (`WebSocketPlugin.dll`). It is responsible for:

- Initializing connections
- Sending and receiving messages
- Routing data to higher-level systems (`GameController`, `NetworkManager`, etc.)
- Managing room logic and reconnections

- `OnMsgReceived_Native`
  - `Inputs: string msg`
  - `Output: void`
  - Purpose: Callback method triggered by the native plugin when a message is received from the WebSocket server.
  - Usage Notes:
    - This must be registered with the plugin via `SetMsgCallback`.

        - Should immediately **queue the message** to be processed on the main thread.

- `Connect`
    - `Inputs: string url`
    - `Output: void`
    - Purpose: Initializes the native WebSocket connection to the specified server URL.
    - Usage Notes:
        - Internally calls `WebSocketPlugin.InitializeWebSocket(url)`.
        - Should be called after registering the callback to avoid missing messages.

- `ConnectToServer`
    - `Inputs: void`
    - `Output: void`
    - Purpose: Wrapper method that sets the message callback and connects to the server using the configured endpoint.
    - Usage Notes:
        - Can include logging, error handling, or load server config dynamically.

- `JoinRoom`
    - `Inputs: string code`
    - `Output: void`
    - Purpose: Sends a message to the server indicating which room or match the client wants to spectate.

- `Send`
    - `Inputs: string msg`
    - `Output: void`
    - Purpose: Sends a raw JSON string to the server through the active WebSocket connection.

- `Disconnect`
    - `Inputs: void`
    - `Output: void`
    - Purpose: Closes the WebSocket connection gracefully.
    - Usage Notes:
        - Should be called on app shutdown or scene unload.

- Internally calls `WebSocketPlugin.CloseWebSocket()`.

- `ProcessMessage`
  - `Inputs: string json`
  - `Output: void`
  - Purpose: Parses and routes incoming JSON messages to the appropriate game system (e.g., updating the game state).

`MainSceneManager.cs`

Responsible for preparing the AR experience by:

- Placing the virtual arena in the real world
- Connecting the user to a multiplayer game room
- Initializing communication with the backend
- Acting as the entry point for the AR duello spectating experience

This script focuses purely on scene and AR-related responsibilities, while gameplay state is handled by `GameController` and networking by `NetworkManager` / `WebSocketManager`.

- `PlaceArena`
  - `Inputs: Vector2 screenPos`
  - `Output: void`
  - Purpose: Tries to place the arena based on detected AR planes using ARFoundation. If no plane is found (e.g. bad lighting, no surfaces), it falls back to placing the arena 2 meters in front of the camera. Once placed, it sets a flag (`arenaPlaced = true`) and links the arena to the `GameController` so it can start tracking and managing the duel.
  - Usage Notes:
    - Instantiates the duel arena prefab at the selected pose.
    - Can include visual feedback (placement indicator, sound).
    - Should disable further placement actions after success.

- `InitializeConnection`
  - `Inputs: void`
  - `Output: IEnumerator`
  - Purpose: Coroutine that initializes the WebSocket connection and waits until the system is ready to join a room.
  - Usage Notes:

- Should start early in the AR scene lifecycle (e.g., `Start()` or after user places arena).
- Can show a loading spinner or connection status UI during the process.

`GameController.cs`

Responsible for managing the entire gameplay loop in the AR duello system.

- `ProcessGameStateUpdate`
  - `Inputs: GameState newState`
  - `Output: void`
  - Purpose: Processes incoming game state updates and reflects them visually in the AR scene.

- `SpawnPlayers`
  - `Inputs: void`
  - `Output: void`
  - Purpose: Instantiates both players into the AR arena based on the arena's transform and scale.

- `SpawnPlayer`
  - `Inputs: PlayerState player, Vector3 position, Quaternion rotation`
  - `Output: void`
  - Purpose: Instantiates a specific player prefab and initializes it with the current state.

- `HandlePlayerStatusChanged`
  - `Inputs: PlayerState player, PlayerController playerController`
  - `Output: void`
  - Purpose: Updates the player visual/animation state based on current gameplay status.

- `EndGame`
  - `Inputs: string winnerId`
  - `Output: void`
  - Purpose: Finalizes the game with winner/loser effects.

PlayerController.cs

- `SetHealth`
  - `Inputs: int newHealth`
  - `Output: void`
  - Purpose: Updates the player's current health, scales the health bar accordingly, and recolors it based on thresholds.

- `TakeDamage`
  - `Inputs: int damage`
  - `Output: void`
  - Purpose: Handles incoming damage, reduces health, and triggers hit feedback.

- `ShowAttackAnimation`
  - `Inputs: int damage`
  - `Output: void`
  - Purpose: Instantiates the attack VFX prefab based on the attack name.

- `SetAvatar`
  - `Inputs: string modelName`
  - `Output: void`
  - Purpose: Changes the player's avatar to the specified model.

6. Test Scenarios

Title: AR Spectator Experience – From Room Code Entry to Duello Conclusion

- **Objective:** To verify the functionality and user experience of the AR Spectator feature in the mobile app, including server room code entry, AR plane detection, arena placement, real-time player visualization, battle updates, and session termination behavior.
- **Actors**:
  - Spectator
  - Server
  - Players

- **Preconditions**:
  - A server instance (duello session) must be active and broadcasting a valid room code.

- The spectator must have a compatible AR-supported mobile device.
- Internet connection must be stable between client and server.
- The spectator must have been registered to the mobile app successfully.

- **Test Steps & Expected Results**
  1) Room Code Entry
     - Step: Spectator opens the AR Spectate page and enters a valid server room code.
     - Expected Result: If the code is valid, the spectator joins the server session. If not, an error prompt appears.
     - *Bonus*: Spectator can scan a QR code instead of entering the room code manually.
  2) Plane Detection
     - Step: Spectator scans the surrounding environment for an AR plane.
     - Expected Result: If a valid surface is detected, the "Show Arena" button becomes active. If not, an error prompt appears.
  3) Arena Placement
     - Step: Spectator taps "Show Arena".
     - Expected Result: Arena is placed on the detected plane. Player avatars are spawned at predefined positions within the arena.
  4) Player Visualization
     - Step: Each spawned player asset displays the correct username and a health bar above it.
     - Expected Result: Health bars dynamically update in real-time based on server data (e.g., damage taken during attacks).
  5) Attack Visualization
     - Step: Server sends action events (e.g., fireball, lightning).
     - Expected Result: Corresponding visual animations are displayed in the AR scene along with damage info on the target player.
  6) Voluntary Exit
     - Step: Spectator attempts to leave before the duello ends.
     - Expected Result: A confirmation popup appears. Selecting "Yes" removes the spectator from the session.
  7) Duello Conclusion
     - Step: One player wins the duello.
     - Expected Result: The spectator sees a prompt showing the winner and loser of the match.
  8) Connection Loss

- Step: Network connection between the spectator and server is interrupted.
- Expected Result: An error prompt informs the user, and they are disconnected from the session.

➢ (*Bonus*) Spectator Info Display

- Step: App displays a panel showing the usernames (and optionally thumbnails) of all connected spectators.
- Expected Result: The panel is updated in real-time as spectators join or leave.

➢ (*Bonus*) Spectator Chat

- Note: This feature is under consideration for future versions.
- Expected Result: Not applicable in current version.

– **Postconditions**:

o The spectator has successfully completed the session or has exited voluntarily.

o The arena and player visuals are cleared after session termination or disconnection.

5.What We've Done

→ **AR Environment Setup:**

o Integrated Unity's ARFoundation framework to enable raycasting for AR object placement.

o Developed features allowing users to place a virtual game arena by tapping anywhere in the world and manipulate it by dragging or pinching.

o Players spawned and animated properly in the arena.

→ **Network Communication:**

o Implemented WebSocket-based communication for real-time multiplayer snychronization.

o Created a robust *NetworkManager* to handle message queuing, latency tracking, and dynamic connection adjustments.

o Ensured seamless room joining and game state updates through the *WebSocketManager*.

→ **Multiplayer Features:**

o Enabled player spawning using *PlayerSpawner*.

o Integrated Photon-like functionality for smooth turn-based gameplay and state sharing.

→ **Game Logic Enhancements:**

- Added core gameplay mechanics (e.g., player actions, attack effects, turn management) using *GameController*.
- Implemented smooth animations for player movements and attack visual effects.

→ **Performance Optimization:**

- Optimized AR rendering to maintain high performance on mobile devices.
- Adjusted network processing rates dynamically based on latency to minimize dropped messages and improve responsiveness.

7. Problems Faced

1. **WebSocket Library Configuration on Windows:**
   a. **Problem:**
   Configuring the WebSocket library (uWebSocket and WebSocket++) on Windows was extremely challenging due to platform-specific dependencies and build requirements.
   b. **Solution:**
      i. Spent significant time resolving compatibility issues with the Windows environment.
      ii. Carefully configured the development environment to meet the library's dependencies and requirements.
      iii. Documented the setup process for future reference to minimize configuration issues.
2. **Real-Time Data Handling:**
   a. **Problem:**
   Designing an efficient system to fetch, process, and display real-time data while ensuring low latency and high reliability was a major obstacle.
   b. **Solution:**
      i. Leveraged WebSocket-based communication for real-time updates.
      ii. Implemented message queuing and latency tracking (via *NetworkManager*) to process data efficiently.
      iii. Designed fallback mechanisms to handle dropped connections and ensure continuity of data flow.

8. Future Work

- **Animation Integration:**

  Add animations for player movements, attacks, and interactions to enhance gameplay immersion.

- **Server Integration:**

  Establish a stable and efficient connection to the game server for real-time data synchronization.

- **Asset and Animation Expansion:**

  Source and integrate diverse 3D model assets and animations to provide variety and improve visual appeal.